

Term Project Report

김종원

Problem 1 CNN Accelerator for SR

1. Baseline code

본 프로젝트의 베이스라인 하드웨어는 hw/cnn_accel에 있는 3-layer ESPCN용 CNN 가속기이다. 입력은 128×128 그레이 스케일 이미지이며, $T_i = T_o = 16$ 으로 병렬 MAC을 수행하고 AHB 버스를 통해 외부 SRAM에서 이미지를 DMA로 읽어 온칩 in_img에 저장한 뒤, 레이어 단위로 convolution-batchnorm-activation을 반복한다. 중간 피쳐맵은 두 개의 DPRAM 뱅크에 ping-pong으로 두고, 마지막 레이어 출력 채널(4채널)은 BMP writer로 덤프 하여 소프트웨어 기준 ofmap과 비교한다.

주요 모듈 구성은 다음과 같다. 상위는 top_system과 top_system_tb이고, 가속기 본체는 cnn_accel이다. 제어는 cnn_fsm이 VSYNC/HSYNC/DATA 타이밍과 픽셀 및 탭 인덱스를 만들고, 연산은 conv_kern-mac_kern-mac과 bnorm_quant_act(+ bias/act shifter)가 담당한다. 가중치 및 scale 및 bias는 SPRAM에 hex로 초기화되며, 이미지 이동에는 dma2buf와 AHB 인터커넥트가 쓰인다. 시뮬레이션 출력은 bmp_image_writer가 담당한다.

베이스라인 테스트 벤치에서 3개 레이어의 대표 설정은 bias_shift가 9/17/17, act_shift가 7/7/7이고, 첫 레이어만 first-layer merge로 is_conv3x3=0, 이후 두 레이어는 3×3 이다. 검증 스크립트 check_hardware_results.m은 out_sw의 L03 ofmap과 하드웨어 BMP를 채널별로 비교한다.

시뮬레이션 결과는 아래 그림과 같다.



```

명령 창

MATLAB을 처음 사용한다면 시작하기를 참조하십시오.

Results of the channel 01 are same!
Results of the channel 02 are same!
Results of the channel 03 are same!
Results of the channel 04 are same!
>>

```

Problem 1의 핵심 과제는 이 베이스라인을 2025Spring 8-layer 네트워크에 맞게 확장하는 것이다. 확장 코드는 hw/cnn_accel_2025Spring에 두었고, 레이어 수 N_LAYER=8, 소프트웨어에서 보낸 설정 및 hex, 마지막 단의 sub-pixel + residual 경로, 그리고 그에 맞는 BMP writer를 포함한다. 아래 절부터는 그 확장에 필요한 참조 소프트웨어와 데이터, 하드웨어 검증을 서술할 예정이다.

2. Reference S/W

참조 소프트웨어는 sw/에 있으며, 양자화된 CNN을 MATLAB에서 수행해 하드웨어와 동일한 입출력 및 파라미터 hex를 만든다. 2025Spring 모델은 hw/uniform_architecture_2025Spring.m을 진입점으로 하고, 네트워크 구조는 arch/arch_2025Spring.m을 run한다. (파일명이 숫자로 시작하는 2025Spring.m은 MATLAB run이 거부할 수 있어 arch_2025Spring 래퍼를 사용한다.) 입력 이미지는 img/butterfly_GT.BMP를 128×128로 쓰고, 가중치는 bin/2025Spring.weights_layer_... 을

로드 한다. 실행 결과로 test_vector_2025Spring.mat, 레이어별 ofmap BMP, 초 해상 결과 등이 생성된다.

(a) Configuration

Table I: 2025Spring Network Architecture for Image Super-Resolution

Layer	Filter size	Input channels	Output channels	Input	Output
1	3×3	1	16	128×128×1	128×128×16
2	1×1	16	16	128×128×16	128×128×16
3	3×3	16	16	128×128×16	128×128×16
4	3×3	16	16	128×128×16	128×128×16
5	3×3	16	16	128×128×16	128×128×16
6	3×3	16	16	128×128×16	128×128×16
7	1×1	16	16	128×128×16	128×128×16
8	3×3	16	4	128×128×16	128×128×4

2025Spring 네트워크는 8개의 convolution 단과 그 뒤의 sub-pixel(픽셀 재배열) 및 residual 합성으로 구성된다. 대략적인 레이어 역할은 다음과 같다. 1번 레이어는 입력 1 채널을 16채널로 올리는 3×3이나, 하드웨어에서는 first-layer merge 때문에 $q_{is_conv3x3}=0$ 으로 두어 한 사이클에 이웃 픽셀을 묶어 처리한다. 2번과 7번은 1×1 convolution이고, 3~6번과 8번은 3×3이다. 8번 레이어만 출력 채널이 4이며 마지막 레이어 플래그와 linear activation을 사용한다. 이후 4채널을 2×2 sub-pixel로 풀어 256×256 고해상도에 가깝게 재배열하고, 저해상도 입력을 업 샘플링한 성분과 더하는 residual 경로가 붙는다.

하드웨어에 넘기는 레이어별 제어 및 시프트 값은 test_vector_2025Spring.mat에서 읽어 테스트 벤치에 넣은 것과 동일하며, 이를 Table II로 정리한다.

시뮬레이션 결과 및 그에 따른 Table2는 아래와 같다.

```

명령 창
MATLAB을 처음 사용한다면 시작하기를 참조하십시오.

[ INFO ] parse and create output buffers
[ INFO ] parse and load params
[ INFO ] forwarding
[ INFO ] layer 0 ( conv ) (scale 00.16) (biases +-07.08) (bias_shift = 16 act_shift = 01)
[ INFO ] layer 1 ( conv ) (scale 01.15) (biases +-07.08) (bias_shift = 22 act_shift = 01)
[ INFO ] layer 2 ( conv ) (scale 00.16) (biases +-07.08) (bias_shift = 23 act_shift = 01)
[ INFO ] layer 3 ( conv ) (scale 00.16) (biases +-07.08) (bias_shift = 23 act_shift = 01)
[ INFO ] layer 4 ( conv ) (scale 00.16) (biases +-07.08) (bias_shift = 23 act_shift = 01)
[ INFO ] layer 5 ( conv ) (scale 00.16) (biases +-07.08) (bias_shift = 23 act_shift = 01)
[ INFO ] layer 6 ( conv ) (scale 00.16) (biases +-07.08) (bias_shift = 23 act_shift = 00)
[ INFO ] layer 7 ( conv ) (scale 00.16) (biases +-07.08) (bias_shift = 24 act_shift = 00)
[ INFO ] layer 8 ( sr_flat )
[ INFO ] layer 9 ( lp_sres )
PSNR for Bicubic Interpolation: 27.43 dB
PSNR for SRCNN Reconstruction: 32.68 dB
>>

```

Table II: 2025Spring's configuration parameters

	Layer							
	#1	#2	#3	#4	#5	#6	#7	#8
q_is_first_layer	1	0	0	0	0	0	0	0
q_is_last_layer	0	0	0	0	0	0	0	1
q_is_conv3x3	0	0	1	1	1	1	0	1
bias_shift	16	22	23	23	23	23	23	24
act_shift	1	1	1	1	1	1	0	0

q_is_first_layer는 레이어 1만 1이고, q_is_last_layer는 레이어 8만 1이다. q_is_conv3x3는 레이어 1 및 2 및 7이 0, 나머지 3×3 구간이 1이다. 특히 레이어 1은 구조상 3×3이지만 merge 스케줄 때문에 플래그를 0으로 두는 점이 중요하다. bias_shift는 16, 22, 이후 대부분 23, 마지막만 24이고, act_shift는 앞쪽 레이어가 1, 7 및 8번이 0이다. 이 값들은 batch-norm과 양자화 시 고정소수점 정렬에 쓰이므로, 소프트웨어와 한 비트라도 어긋나면 ofmap이 크게 달라진다. LAYER_CONFIG 워드에는 first/last/conv3x3/act_type과 layer_index, bias_shift, act_shift가 비트 필드로 패킹 된다.

(b) Data preparation

소프트웨어 추론 이후에는, write_cnn_model_to_hex_file.m으로 하드웨어용 hex를 만든다. 입력 이미지는 8 bit 및 32 bit 형태로 저장되고, 가속기는 주로 butterfly_32bit.hex를 DMA로 읽는다. 가중치는 레이어 1에 export_weight_to_file_merge, 이후 레이어에 export_weight_to_file_reorder를 적용한 뒤 all_conv_weights.hex로 합친다. scale과 bias도 레이어별로 뽑아 all_conv_scales.hex, all_conv_biases.hex로 모은다. 이 파일들을 hw/cnn_accel_2025Spring/input_data/(및 필요 시 img/)로 복사하고, 검증용으로 마지막 레이어 ofmap BMP를 out_sw/에 둔다.

아래는 시뮬레이션 결과 및 그에 따른 Table3이다.

```

명령 창
MATLAB을 처음 사용한다면 시작하기를 참조하십시오.

The model is 2025Spring !!!

Exporting layer 1 ..... done!
Exporting layer 2 ..... done!
Exporting layer 3 ..... done!
Exporting layer 4 ..... done!
Exporting layer 5 ..... done!
Exporting layer 6 ..... done!
Exporting layer 7 ..... done!
Exporting layer 8 ..... done!
>>

```

Table III: The buffer requirements

	No. of bit per line	Number of lines								Buffer Size in total	
		#1	#2	#3	#4	#5	#6	#7	#8	Word (bit)	No. of words
Weight	128	16	16	144	144	144	144	16	144	128	768
Scale	16	16	16	16	16	16	16	16	16	16	128
Bias	16	16	16	16	16	16	16	16	16	16	128

Table III에 정리한 weight, scale, bias 버퍼 크기가 최적인지 검토한 결과는 다음과 같다. Scale과 Bias는 레이어마다 출력 채널 수를 $T_o (=16)$ 에 맞춰 패딩 하여 저장하므로 레이어당 16워드가 필요하고, 8개 레이어를 합치면 총 128워드가 된다. 이는 실제 필요량과 일치하므로 Scale/Bias 버퍼는 최적에 가깝다고 볼 수 있으며, 워드 당 16 bit를 기준으로 하면 각각의 총 용량은 $128 \times 16 = 2,048$ bit이다.

이에 비해 Weight는 레이어 종류에 따라 필요한 워드 수가 달라진다. 첫 번째 레이어는 first-layer merge 방식으로, 두 번째와 일곱 번째 레이어는 1×1 convolution으로 인해 각각 16워드만 사용하는 반면, 나머지 3×3 convolution 레이어는 각각 144워드를 사용한다. 이를 모두 합하면 768워드가 되고, 워드 당 128 bit를 곱하면 $768 \times 128 = 98,304$ bit가 되어 hex 파일에 실제로 저장된 용량과 일치한다. 따라서 Table III의 weight 버퍼 크기(768워드)는 필요 하한에 해당한다.

다만 하드웨어에서 모든 레이어를 3×3 으로 가정하여 $N_{LAYER} \times (T_i \times T_o \times 9)/N = 8 \times 144 = 1152$ 워드로 할당하면, 실제 사용량인 768워드보다 384워드가 남는 과대 할당이 발생한다. 이러한 방식은 구현을 단순하게 만드는 장점은 있으나 메모리 효율 측면에서는 최적이지 않다.

결론적으로 Scale/Bias 버퍼는 최적에 가깝지만, Weight 버퍼는 레이어별 실제 hex 라인 수의 합인 768워드가 더 효율적이며 전 레이어를 3×3 으로 통일한 1152워드 할당은

suboptimal하다는 결론을 도출했다.

4. H/W simulation and verification for 2025Spring

2025Spring 하드웨어는 hw/cnn_accel_2025Spring에서 N_LAYER=8로 두고, Table II의 시프트 및 플래그를 레이어 루프마다 기록한 뒤 DMA로 이미지를 적재하고 여덟 레이어를 순차 실행한다. 가중치 베이스 주소는 1×1이면 To워드, 3×3이면 To×9워드씩 진행하고, scale/bias 베이스는 레이어마다 To씩 진행한다. 클럭은 100 MHz 고정, (주기 10 ns), 해상도는 128×128이다.

마지막 레이어 이후에는 초 해상 출력을 위해 sub-pixel과 residual 경로가 필요하다. 마지막 convolution의 4채널 잔 차와, 저해상 입력 in_img를 더해 [0,255]로 클립 한 뒤 네 바이트를 하나의 32 bit 픽셀로 포장한다. bmp_image_writer_32bit는 이 스트림을 받아 sub-pixel 배치에 맞게 풀어 256×256 이미지(convout_hr.bmp)을 만들고, 동시에 채널 별 8 bit BMP(convout_ch01~04)도 남긴다. 디버그용 대량 \$fwrite들은 시뮬레이션 시간 절약을 위해 비활성화하였다.

시뮬레이션 결과는 아래 그림들과 같다.



```
명령 창
MATLAB을 처음 사용한다면 시작하기를 참조하십시오.
Results of the channel 01 are same!
Results of the channel 02 are same!
Results of the channel 03 are same!
Results of the channel 04 are same!
>>
```

기능 검증은 check_hardware_results.m으로 진행하였다. 소프트웨어 기준은 out_sw/ofmap_L08_ch01~04.bmp이고, 하드웨어 출력은 out/convout_ch01~04.bmp이다. 채널마다 절대차의 최댓값이 0이면 하드웨어가 참조 소프트웨어와 동일하게 동작한다고 본다.

구현 및 포팅 과정에서 Vivado 시뮬레이션을 위해 몇 가지 정합이 필요했다. generate 루프에서 scale[i]처럼 잘못된 부분 선택을 쓰면 폭 불일치가 나므로 scale[(i*PARAM_BITS)+:PARAM_BITS] 형태의 패킹 인덱싱을 사용해야 한다는 점을 알게 되었다. 또한, out_pixel/out_valid가 top까지 연결되어 테스트 벤치와 일치해야 하며, 2025Spring용 32 bit BMP writer stub가 비어 있지 않고 실제 서브 픽셀 재배열을 수행해야 한다. 이러한 수정 후 시뮬레이션과 MATLAB 비교로 2025Spring 가속기의 기능 동등성을 확인한다.

Problem 2: Optimization

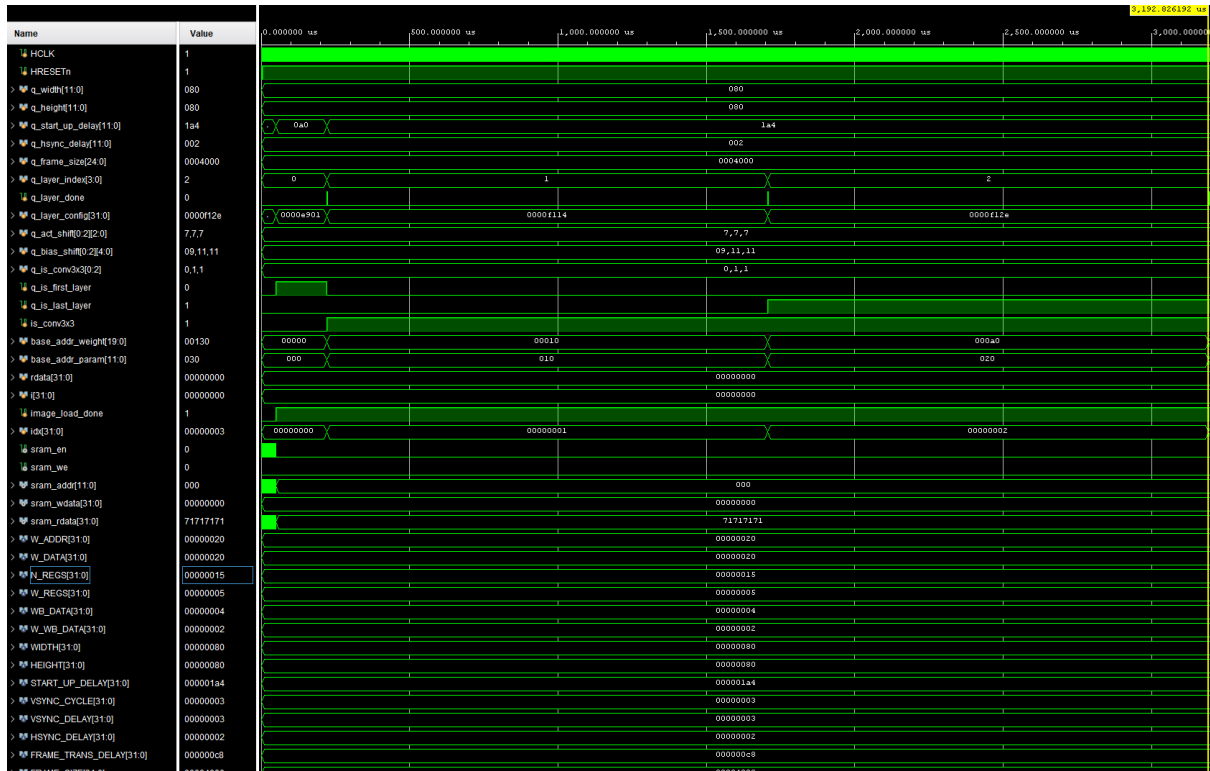
1. Optimization

(a) Problem and scopes

베이스라인 CNN 가속기(cnn_accel)는 128×128 입력에 대해 3개 레이어 ESPCN을 $T_i = T_o = 16$, 동작 주파수 100 MHz로 수행한다. 과제에서 제시한 베이스라인 성능은 실행 시간 $t_0 = 3930 \mu s$, 온칩 버퍼 $s_0 = 4280 \text{ Kbits}$ 이며, 전체 개선도는 $S_{overall} = (t_0/t) \times (s_0/s)$ 로 평가한다. 여기서 클럭 주파수, T_i/T_o , 입출력 해상도는 변경할 수 없으므로, 개선은 데이터 이동 및 제어 공백 및 버퍼 구조를 바꾸는 쪽으로만 가능하다.

베이스라인의 버퍼 구성을 분해하면 대략 입력 이미지 버퍼 128 Kbits, 피쳐맵 더블 버퍼 $2048 \times 2 = 4096 \text{ Kbits}$, 가중치, bias/scale 약 55 Kbits 수준으로, s_0 의 대부분이 피쳐맵 두 장에 묶여 있다. 시간 측면에서는 라인마다 긴 HSYNC 공백(약 160 사이클)과, 레이어 3에서 Cout=4인데 $T_o=16$ 인 점 때문에 커널 idle 비율이 큰 것이 병목으로 알려져 있다. 본 최적화는 hw/cnn_accel_opt에서 위 제약을 지키면서 실행 시간과 버퍼를 동시에 줄이는 것을 범위로 한다.

아래는 베이스라인에 대한 시뮬레이션 결과이다.



```

명령 창

MATLAB을 처음 사용한다면 시작하기를 참조하십시오.

Results of the channel 01 are same!
Results of the channel 02 are same!
Results of the channel 03 are same!
Results of the channel 04 are same!
>>
  
```

(b) Optimization methods

강의에서 제시된 방향은 (1) 입력 재배치와 DMA 트랜잭션 수 감소, (2) DMA와 convolution 파이프라이닝으로 입력 버퍼 축소, (3) Layer 3에서 convolution 커널 완전 활용, (4) layer fusion이다. 본 설계는 기능 동등성(check_hardware_results.m 일치)을 깨지 않는 선에서 효과가 큰 항목을 우선 적용하였고, 아래는 실제 수정한 코드이다. 주요 변경 파일은 cnn_accel.v, cnn_fsm.v, top_system.v, top_system_tb.v이다.

첫째, 입력 DMA 패킹(4픽셀/워드). AHB 32 bit에 맞게 butterfly_32bit_reorder.hex를 쓰고, 라인당 전송 횟수를 WIDTH/4로, 주소 증가를 WIDTH 바이트로 두었다. valid마다 in_img에 네 바이트를 쓴다.

테스트 벤치에서 패킹 hex와 워드 수:


```
// top_system_tb.v
parameter N_WORD = WIDTH * HEIGHT / 4;
parameter INIT_FILE = "img/butterfly_32bit_reorder.hex";
parameter N_CELL = 304; // weight words: 16+144+144
```

DMA 전송 횟수 및 라인 stride (cnn_accel.v):

```
parameter PIX_PER_WORD = 4;
// ...
num_trans_ld = q_width / PIX_PER_WORD; // 128 -> 32 beats/line
// ...
start_addr_ld <= start_addr_ld + q_width; // byte stride (not WIDTH*4)
```

픽셀 언팩 저장:

```
if(data_vld_o_ld) begin
    in_img[in_pixel_count + 0] <= data_o_ld[ 7: 0];
    in_img[in_pixel_count + 1] <= data_o_ld[15: 8];
    in_img[in_pixel_count + 2] <= data_o_ld[23:16];
    in_img[in_pixel_count + 3] <= data_o_ld[31:24];
end
```

둘째, HSYNC 축소와 라인 버퍼 prefetch. DATA 중에 다음 하단 행을 lb_next로 읽어 두고, HSYNC에서는 윈도우만 한 줄 밀어 올려 HSYNC_DELAY=2로 두었다.

```
// top_system_tb.v - 레이어마다 delay 기록
q_start_up_delay = q_is_first_layer ? 12'd160 : 12'd420;
q_hsync_delay = 12'd2;
task_AHBwrite(`CNN_ACCEL_DELAY_PARAMS, {q_hsync_delay, q_start_up_delay});
```

fmap 읽기: VSYNC 때 초기 2줄 적재, DATA 때 row+2 prefetch.

```
// cnn_accel.v
wire lb_load_issue = ... && (lb_load_cnt < LB_PRELOAD_WORDS); // 2*WIDTH
wire lb_pref_issue = ... && ctrl_data_run
    && (lb_pref_cnt < q_width) && ((row + 2) < q_height);
// ...
fmap_buf_addrb = (row + 2) * q_width + lb_pref_cnt; // prefetch
```

HSYNC에서 시프트 및 설치:

```
if(!q_is_first_layer && ctrl_hsync_run && (ctrl_hsync_cnt == 0)) begin
    lb_pref_cnt <= 0;
    if(row >= 1) begin
        for(gi=0; gi<2*WIDTH; gi=gi+1)
            sw_linebuf[gi] <= sw_linebuf[gi+WIDTH];
        for(gi=0; gi<WIDTH; gi=gi+1)
            sw_linebuf[2*WIDTH+gi] <=
                ((row+1) < q_height) ? lb_next[gi] : {To*ACT_BITS{1'b0}};
    end
end
```

셋째, 단일 fmap + 3줄 라인 버퍼(부분 layer fusion). 두 번째 DPRAM을 삭제하고 u_fmap_buff 하나만 남겼다. L2/L3 din은 sw_linebuf에서 읽는다.

```
localparam LINE_BUF_WORDS    = 3 * WIDTH;    // n = 3
localparam LB_PRELOAD_WORDS  = 2 * WIDTH;    // row0: [0, fmap0, fmap1]
reg [To*ACT_BITS-1:0] sw_linebuf [0:LINE_BUF_WORDS-1];
reg [To*ACT_BITS-1:0] lb_next    [0:WIDTH-1];

wire fmap_wr_en = vld_o[0] && !q_is_last_layer;
dpram #(.W_DATA(To*ACT_BITS), .N_WORD(FRAME_SIZE))
u_fmap_buff(    );
```

라인 버퍼에서 3x3 탭을 읽는 경로(L2/L3 공통):

```
// boundary pad + sw_linebuf[tap_r*WIDTH + col]
boundary = (is_first_row && (tap_r==0)) || (is_last_row && (tap_r==2))
           || (tap_c < 0) || (tap_c >= q_width);
lb_idx = tap_r*WIDTH + ((tap_c < 0) ? 0 : ((tap_c >= WIDTH) ? (WIDTH-1) :
tap_c));
din_tap0 = boundary ? {Ti*Wl{1'b0}} : sw_linebuf[lb_idx];
```

넷째, Layer 3 multipixel. 픽셀 4xOC 4 매핑은 마지막 행 끝 픽셀 불일치로 최종본에서
고고 stride=1을 유지했다(cnn_fsm.v의 col_stride = 1). 시간 이득은 DMA 및 HSYNC에,
버퍼 이득은 fmap 단일화에 두었다.

다섯째, 가중치 SPRAM 적정화.

```
// cnn_accel.v - was 3*144=432  
parameter N_CELL = 304; // 16 + 144 + 144
```

이상으로 수정 코드의 핵심은 (i) DMA 패킹 및 stride, (ii) HSYNC=2 + prefetch/shift, (iii) fmap 1뱅크+sw_linebuf/lb_next, (iv) N_CELL=304이다.

수정한 파일은 다음과 같다: cnn_accel.v, cnn_fsm.v, bmp_image_writer.v, top_system.v, top_system_tb.v.

(c) Evaluation

기능 검증으로, Vivado에서 top_system_tb를 시뮬한 뒤 out/convout_ch01~04.bmp를 얻고 check_hardware_results.m으로 out_sw의 L03 ofmap과 비교한다. 네 채널 모두 max_diff==0이어야 베이스라인과 동일하게 동작한다고 본다.

이어서 과제에서 요구한 실행 시간 t 와 버퍼 크기 s , 그리고 전체 개선도 지표를 보고한다. 전체 개선도는 다음 식으로 정의된다.

$$S_{overall} = (t_0 / t) \times (s_0 / s)$$

여기서 t_0 , s_0 는 각각 베이스라인의 실행 시간과 버퍼 크기이고, t, s 는 최적화 설계의 값이다. 과제에서 제시한 베이스라인 값은 $t_0 = 3930 \mu s$, $s_0 = 4280 \text{ Kbits}$ 이다.

본 설계의 온칩 버퍼 s 는 선언된 저장 요소의 합으로 계산한다. in_img 128 Kbits, fmap 단일 뱅크 2048 Kbits, 3줄 라인 버퍼(sw_linebuf) 48 Kbits, prefetch 줄(lb_next) 16 Kbits, weight 약 39 Kbits, bias/scale 약 1.5 Kbits를 더하면

$$s = 128 + 2048 + 48 + 16 + 39 + 1.5 = 2280.5 \text{ Kbits}$$

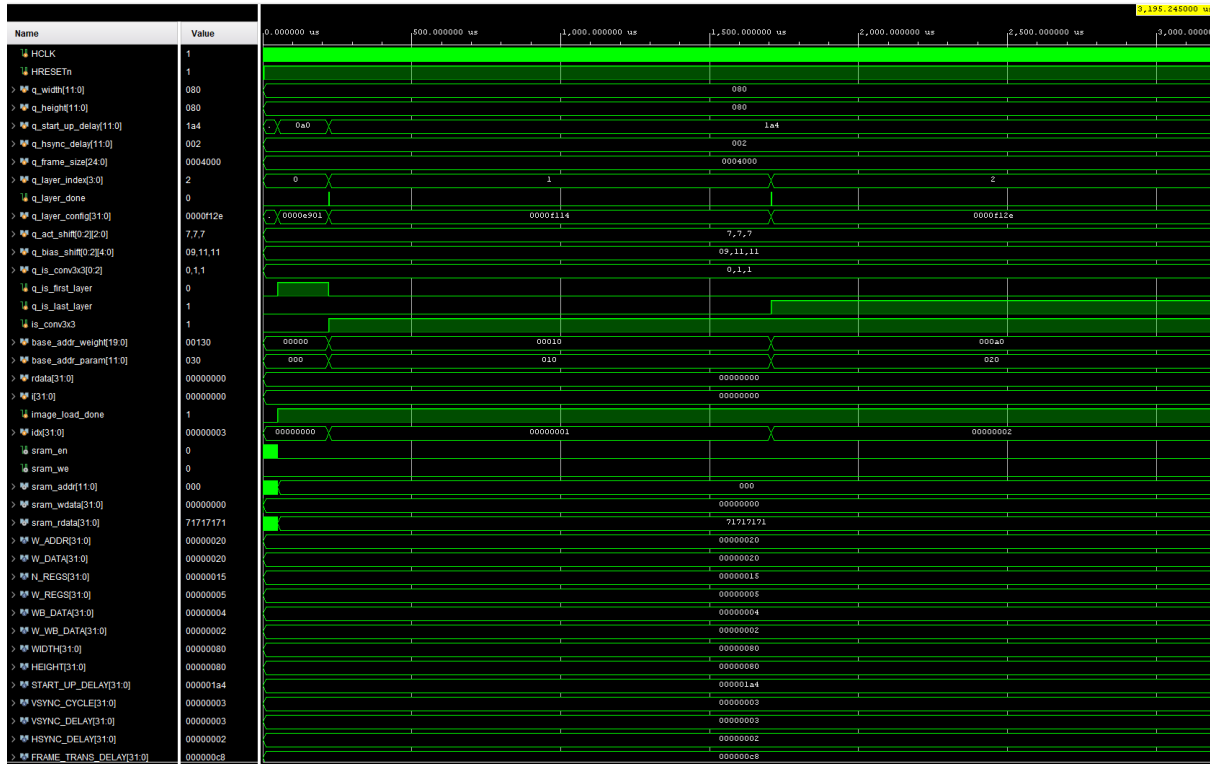
이므로 버퍼 쪽 이득은 $s_0/s = 4280/2280.5 \approx 1.877$ 이다.

실행 시간 t 는 테스트 벤치 출력으로 측정하였다. 시뮬레이션 종료 시 OPT DONE: t = 3194 us(sim time 3193965000)가 관측되었으므로 $t = 3194 \mu s$ 이다. 이를 대입하면

$$S_{overall} = \frac{t_0}{t} \times \frac{s_0}{s} = \frac{3930}{3194} \times \frac{4280}{2280.5} \approx 1.230 \times 1.877 \approx 2.309$$

이다. 즉 시간 측면에서 약 1.23배, 버퍼 측면에서 약 1.88배 개선되어, 전체 지표는 약 2.31이다.

아래 그림들은 계산에 사용한 지표 및 시뮬레이션 결과이다.



```
=====
OPT DONE: t = 3194 us (sim time 3193965000)
Buffer s (Kbits) = 128(in)+2048(fmap)+48(linebuf)+16(lb_next)+39(w)+1.5(p) ~= 2280.5
S_overall = (3930/t)*(4280/s); see OPT_RESULTS.txt
=====
```

```
명령 창
MATLAB을 처음 사용한다면 시작하기를 참조하십시오.

Results of the channel 01 are same!
Results of the channel 02 are same!
Results of the channel 03 are same!
Results of the channel 04 are same!
>>
```

2. Optimality analysis

라인 윈도우 $n=3$ 을 고른 이유는 3×3 이 한 출력 행에 입력 세 줄을 필요로 하기 때문이다. $n < 3$ 이면 부족하고 $n > 3$ 이면 SRAM만 늘어난다. 두 번째 전체 fmap 대비 수십 Kbits로 같은 역할을 하므로 s_0 / s 에 가장 직접적이다. 참고로 과제에서 예시로 든 입력 버퍼를 n 줄만 두고 DMA와 convolution을 파이프라인하는 방식은 적용하지 않았다. 본 설계의 $n=3$ 은 입력(in_img)이 아니라, 3×3 연산에 필요한 피쳐맵 주변 줄(위, 가운데, 아래)을 유지하기 위한 라인 버퍼 높이이다. in_img는 여전히 한 프레임 전체를 저장한다.

첫 행은 [패드, row0, row1]이어야 하므로 초기 적재는 $2 \times \text{WIDTH}$ 만 읽고 상단을 0으로 둔다. [row0,1,2]로 채우면 중심이 밀려 상단 행이 깨진다. $\text{START_UP}=420$ 은 가중치(최대 144)와 이 적재(256)를 덮는다.

$\text{HSYNC}=2$ 는 DATA 중 prefetch와 짝을 이룬다. prefetch 없이 HSYNC에서 128워드를

읽으면 공백이 다시 커진다. DMA 패킹 인수 4는 32 bit 버스와 8 bit 픽셀의 비이다. $N_CELL=304$ 는 weight hex의 필요 하한이다.

또한, 본 설계의 MAC 유틸리티는 피크 병렬도와 레이어별 유효 출력 채널의 비로 평가할 수 있다. 가속기는 $T_i=T_o=16$ 으로 동작하므로 이론상 피크는 $T_i \times T_o=256$ MAC/cycle 이다. Layer 1,2는 출력 채널이 T_o 에 맞춰져 있어 DATA 구간에서는 이 피크에 가까운 유효 연산을 유지한다. 반면 Layer 3은 Cout=4인데 하드웨어는 여전히 $T_o=16$ 커널을 돌리므로, 유효 MAC은 대략 $256 \times (4/16)=64$ MAC/cycle이고 utilization은 약 25%에 그친다. 즉 L3에서는 커널의 3/4가 사실상 유향이다. Layer 3 multipixel(픽셀 그룹 $\times$ OC 재 매핑)을 적용하면 이 유향을 메워 L3 utilization을 크게 끌어올릴 수 있으나, 마지막 행 끝 픽셀에서 기준 ofmap과 불일치가 남아 최종 설계에서는 비활성으로 두었다. 따라서 현재 구현의 L3 MAC utilization은 베이스라인과 동일하게 약 25%이며, 보고한 실행 시간 개선 ($t_0/t \approx 1.23$)은 MAC 수 증가가 아니라 DMA 패킹과 HSYNC 공백 축소에서 비롯된 것이다. 그리고, 입력 버퍼를 n줄만 남기는 DMA-L1 파이프라인은 줄일 양이 128 Kbits뿐이라 fmap 뱅크 제거보다 우선순위가 낮다.

정리하면 핵심 파라미터는 $n=3$ (+prefetch 1줄), HSYNC=2, DMA $\times$ 4, $N_CELL=304$ 이며, 실측 기준으로 버퍼 $s = 2280.5$ Kbits, 실행 시간 $t = 3194 \mu s$, $S_{overall} \approx 2.309$ 를 얻었다.

아래 표는 최적화를 수행함에 따라 측정된 결과를 정리한 것이다.

측정 결과 요약

항목	베이스라인	최적화
실행 시간 t	3930 μs	3194 μs
버퍼 크기 s	4280 Kbits	2280.5 Kbits
t_0/t	-	≈ 1.230
s_0/s	-	≈ 1.877
$S_{overall}$	-	≈ 2.309